C Dersleri Önemli Notlar

Recursive Functions

- Temel durum ve Çağrı olmak üzere 2 durumdan oluşur.
    - Temel Durum: Fonksiyonun sonlanacağı durumdur.
    - Çağrı durumu: Fonksiyonun sonlanana kadar döndürüleceği döngü durumudur.

```
#include <stdio.h>

int fibonacci(int n){
    if(n==0){
        return 0;
    }
    else if(n==1){
        return 1;
    }
```

```
    return fibonacci(n-1)+fibonacci(n-2);
}
```

Random Fonksiyonu

- Random fonksiyonu için 3 kütüphaneye ihtiyaç duyulur.

```
#include <stdio.h>
#include <stdlib.h>  //rand() ve srand() fonksiyonları için gereklidir.
#include <time.h>    //time() fonksiyonu için gereklidir.
```

❗ srand(time(NULL)); fonkosiyonunun döngü içinde olmasına gerek yoktur. Başta yazılmış olsa yeter.

Pointerler

Göstergeleri Parametre Olarak Yollamak

- C dilinde fonksiyon parametreleri değer parametresi (call by value) veya kaynak parametresi (call by reference) olarak gönderilebilir.
    - **Değer Parametresi (Call by Value):** Fonksiyona değişkenin bir kopyasını göndererek onun üstünde işlem yapar ve orijinal değişkene bir şey olmaz.
    - **Kaynak Parametresi (Call by Referece):** Fonksiyona değişkenin adresini göndererek o adres üzerinden orijinal değişkenle işlem yapar ve orijinal değişkeni etkiler.

Arrays

```
dizi     //Pointer gibi davranır.
dizi[0]  //Değişken gibi davranır.
```

```
&dizi[1] //Pointer gibi davranır.
```

Diziyi Fonksiyona Gönderme

- Dizi fonksiyona gönderildiğinde kaynak parametresi (call by reference) gibi davranır.

```
#include <stdio.h>

void function(int a[],int n);

int main(void){
    int i,data[]={1,2,3};
    function(data,i); //Dizi fonksiyonda yazılırken sadece adı yazıldı.
    return 0;
}

void function(int a[],int n){
    comment;
    comment;
    comment;
}
```

Diziyi Pointer Olarak Atama

```
#include <stdio.h>

void topla(int*);

int main(void){
    int data[]={1,2,3,4};
    printf("%d",topla(data));   //Dizi fonksiyonda yazılırken sadece adı yazıldı.
    return 0;
}
```

```
void topla(int *ptr){
    int i,toplam=0;
    for(i=0;i<4;i++){
        toplam+=*(ptr+i);
    }
return 0;
}
```

2 Boyutlu Diziler

```
int data[satır][sütun]; //Yapısına sahiptir.
```

> ❗ Bir boyutlu dizilerde eleman bulunurken dizi tanımında eleman sayısı bulunmasına gerek yoktur ancak iki boyutlu dizilerde en azından sütun sayısı bulunmalıdır.

```
data[][]={1,2,3,4};   //Yanlış.
data[][];             //Yanlış.
data[][4]={1,2,3,4};  //Doğru.
```

- İki boyutlu diziler ayrıca aşağıdaki gibi yazılabilir.

```
data[3][2]={{1,2},{3,4},{5,6}};
```

2 Boyutlu Diziyi Fonksiyon Parametresi Olarak Alma

> ❗ Fonksiyonu ve prototipini oluştururken içinde en azından sütun sayısı yazmak zorundadır.

```
#include <stdio.h>

int sum(int a[][3]);

int main(void){
    int data[][3]={1,2,3,4};
    printf("%d",sum(data));
    return 0;
}

int sum(int dizi[][3]){
    int i,j,sum=0;
    for(i=0;i<2;i++){
        for(j=0;j<3;j++){
            sum+=dizi[i][j];
        }
    }

    return sum;
}
```

- Şekilde görüldüğü gibi data 2 boyutlu dizisi fonksiyonun içinde direkt yazılmıştır.

Önemli Notlar

- data[n]=x; yazarken n negatif olamaz, tersten başlamak gibi bir durum yoktur.
- a[0]='0' şeklinde int veri tutan bir diziye atama yapıldığında 0'ın ASCII değerini tutar yani syntax doğrudur.
- Dizilerde global değişken olmadığı sürece sadece eleman sayısı verilerek oluşturulan dizilerin elemanları otomatik sıfır olmaz. Örneğin **int data[2][20]={0};** şeklinde yazılmalıdır. Ancak dediğim gibi global değişken tanımlanırsa otomatik sıfır olur.

- Bir array'ın eleman sayısını bulmak için bellekte kapladığı alan/bir elemanın bellekte kapladığı alan yapabiliriz. Örnek:

```
int data[]={1,2,3,4,5,6};
int adet=sizeof(data)/sizeof(int);
```

- **const int x;** gibi bir kod ile değişkeni değiştirilemez hale getirmiş oluruz.

Strings (Dizgiler)

- Bir dizginin sonu **'\0'** ile biter.
  - **char data[20];** gibi bir dizi 19 karakter alır çünkü son karakter '\0' olmalıdır.
- Çıktı alırken **printf("%s",data);** yazılabilir.
- Dizi yerine pointer de kullanılabilir. Örnek:

```
char *pointer="Riceb";
printf("%s",pointer);
```

- Scanf ile string almaya çalıştığımızda varsa boşluğa yoksa enter'e kadar okur. Örnek:

```
char kelime[20];
scanf("%s",kelime);
```

- Gets() fonksiyonu olmadan scanf ile cümle tutabilmek için aşağıdaki komut kullanılabilir. (Cümle sonunda nokta olmak zorunda.)

```
char data[100];
int i=0;

printf("Lutfen bir cumle giriniz:\n");

do{
```

```
    scanf("%c",&data[i]);
    i++;
}while(data[i-1]!='.');
```

Hazır String Fonksiyonları

strlen()

- Yazdığımız string dizisinin uzunluğunu (eleman sayısını) ölçmemize yarar.

```
char kelime[]="Merhaba";
strlen(kelime) //7 sonucunu verir ('\0' karakterini saymaz.)
```

strcat()

- İki adet dizgiyi birleştirmek için kullanılır.
- 2 adet stringi birleştirip ilk yazılana atar bu yüzden ilkinin ikisinin birden tutacak yerinin olması gerekir.

```
char w1[100]="Hello ";
char w2[]="World";

strcat(w1,w2);
printf("%s",w1); //Hello World
```

strncat()

- İlk dizgiye ikinciden n kadar karakter ekler.

```
strncat(d1,d2,n);
```

❗ strncpy aksine sonuna null karakterini ekler.

strcpy()

- Fonksiyona ilk girilen dizginin içine ikincideki datayı kopyalar.

```
char w1[100]="Hello Word";
char w2[100];

strcopy(w2,w1);
printf("%s",w2); //Hello World
```

- Eğer içine kopyaladığımız dizgide bir data varsa o datayı siler.
- Eğer belirli bir yerden sonrası kopyalanmak isteniyorsa o karakter alınıp adresi verilmelidir:

```
char a[50]="C Dersi:Cozumlu Problem Kitabi";
char b[50];

strcpy(b,&a[8]);   //Cozumlu Problem Kitabi
```

strncopy()

- 2.diziden 1.diziye n karakter kadar kopyalar. Sona null karakterini koymaz, onu bizim koymamız gerekir.

```
#include <string.h>
strncpy(d1,d2,n);
d1[i]='\0'
```

- strcpy bunun aksine sona null karakterini ekler.

> ❗ strncpy() fonksiyonları içi dolu bir dizinin içine kopyalama yaparken en baştan başlayarak kopyalayacağı eleman kadarını değiştirerek kopyalar. strcpy'de böyle bir şey yoktur, o tamamen ilk dizinin içini boşaltır.

strcmp()

- 2 dizginin birbirine eşit olup olmadığını karşılaştırmak için kullanılır.

```
char w1[]="Hello World";
char w2[]="Hello World";
strcmp(w1,w2);
```

- Eğer w1<w2 ise pozitif,
  - eğer w1>w2 ise negatif,
  - eğer w1=w2 ise sıfır döner.

strncmp()

- Seçilen karakter sayısı kadar karşılaştırılır.

```
strncmp(d1,d2,n);
```

gets()

- gets fonksiyone scanf() fonksiyonunun aksine entere kadar okur.
- Null karakterini ('\0') otomatik sona koyar. Yani n karakterli bir diziye atama yaparken eğer bütün dizi doldurulursa null karakterini koyamayacağı için ileride sıkıntılar yaşanır.

```
char cumle[50];
gets(cumle);
printf("%s",cumle);
```

puts()

- Sadece adres alır. (String dizileri gibi.)
- String yazdırmak için kullanılır.
- '\0' gördüğünde bunu \n'e çevirir yani sonda alt satıra iner.

```
char cumle[]="Merhaba Dunya";
char kelime[]="Selamlar";
puts(cumle);
puts(kelime);

    /*
      Çıktı:
      Merhaba Dunya
      Kelime
    */
```

putchar()

- Harf harf ekrana yazdırmak için kullanılır.

```
char *ptr='A';
putchar(ptr);
```

- puts() fonksiyonunu aksine sona null karakterini koymaz çünkü dizi değildir.

sscanf()

- Bir string verisini ayrıştırmak için kullanılır.
- Eğer işlem başarılı olursa ayrıştırılan öğe sayısını döndürür.
- Eğer işlem başarısız olursa 0 ya da EOF döner.
- sscanf() fonksiyonu boşlukları ayraç olarak görür.
- **sscanf(dizi,”%format %format”,&variable,&variable);**
- scanning yapıldığı için ampersand kullanmaya dikkat et.
  - str içindeki veriyi okur
  - formatlara göre ayrıştırır.
  - parçalanan veriyi belirtilen değişkenlere aktarır.

```c
#include <stdio.h>

int main() {
    char input[] = "42 3.14 Merhaba";
    int sayi;
    float ondalik;
    char kelime[20];

    sscanf(input, "%d %f %s", &sayi, &ondalik, kelime);

    printf("Sayı: %d\n", sayi);
    printf("Ondalık: %.2f\n", ondalik);
    printf("Kelime: %s\n", kelime);

    return 0;
}

/*
        ÇIKTI:
        Sayı: 42
        Ondalık: 3.14
        Kelime: Merhaba
*/
```

- sscanf() içine yazılan formatların arasında ne varsa ona göre ayırır. Eğer boşluk varsa boşluk, eğik çizgi varsa eğik çizgi, tire varsa tire.

```c
#include <stdio.h>

int main() {
    char tarih[] = "2024-12-23";     //Dizi syntaxsına dikkat et.
    int yil, ay, gun;

    sscanf(tarih, "%d-%d-%d", &yil, &ay, &gun);    //sscaf syntaxına dikkat et.
```

```
    printf("Yıl: %d, Ay: %d, Gün: %d\n", yil, ay, gun);

    return 0;
}

/*
        ÇIKTI:
        Yıl: 2024, Ay: 12, Gün: 23
*/
```

sprintf()

- sscanf() fonksiyonunun tam tersine ayrı olanları birleştirir.
- **sprintf(toplam_dizi,"Yazi %format yazi %format yazi",degisken,degisken);**

```
#include <stdio.h>

int main() {
    char buffer[50];
    int sayi = 42;
    float ondalik = 3.14;

    sprintf(buffer, "Sayi: %d, Ondalik: %.2f", sayi, ondalik);

    printf("Buffer içeriği: %s\n", buffer);

    return 0;
}

/*
        ÇIKTI:
        Buffer içeriği: Sayı: 42, Ondalık: 3.14
*/
```

❗ Özünde printf() olduğu için ampersand kullanılmaz.

strstr()

- İlk girilen dizinin içinde ikinciyi arar.
- Eğer bulursa bulunanın başlangıç adresini döner,
  - eğer bulamazsa null döner.
- Büyük küçük harf duyarlılığı vardır.
- Eğer ikinci dizgi boşsa (“”) ilkinin ilk adresini döner yanı tamamını kabul eder.
- Eğer strstr() bir pointere atanır ve pointer içeriği yazdırılırsa bulunduktan sonrasının tamamı yazdırılır.

Dinamik Bellek

❗ <stdlib.h> kütüphanesi gerekir.

malloc()

```
int *ptr;
ptr=(int *)malloc(eleman_sayisi*sizeof(int));
```

- Aslında malloc adres döner ve bu adresi pointer’e atar.
- Eğer bellek tahsisi başarısızsa null dönecektir.
- İçindeki değerler rastgele olarak belirlenir.

calloc()

- Malloctan faklı olarak içindeki değerleri sıfır olarak atar.
- Syntax farkı olarak da önce eleman sayısını sonra eleman değerini alır.

```
int *ptr;
ptr=(int *)calloc(eleman_sayisi,sizeof(int));
```

realloc()

- önceden tahsis edilmiş bellek alanını değiştirmek için kullanılır.

```
ptr=(int *)realloc(ptr,eleman_sayisi*sizeof(int));
```

- Syntaxtaki değişikliğe dikkkat et.

free()

- Kullandıktan sonra bellek alanını serbest bırakmak için kullanılır.

Karakter Fonksiyonları

Kütüphane:

- Karakter fonksiyonlarını çalıştırmak için <ctype.h> kütüphanesine ihtiyaç duyarız.

Fonksiyonlar:

isalnum()

- Karakterin alfanumerik olup olmadığını kontrol eder. (Harf veya rakam.)
- Eğer rakam ya da harf ise true, değilse false döndürür.

isalpha()

- Harf kontrolü yapar

islower()

- Karakterin küçük olup olmadığınını kontrol eder.

isupper()

- Karakterin büyük olup olmadığını kontrol eder.

isdigit()

- Karakterin rakam olup olmadığını kontrol eder.
- Tek karakter baktığı için rakam bakmak zorunda.

isxdigit()

- Karakterin hexadecimal olup olmadığını kontrol eder.

isspace()

- Boşluk, tab veya kaçış karakteri (\n etc.) olup olmadığını kontrol eder.

isblank()

- Sadece boşluk ve tab kontrolü.

> ❗ "is" ile başlayan fonksiyonlar doğru ise 1, yanlış ise 0 döndürür. Bu sayede if içinde direkt kullanılabilir.

tolower()

- Karakteri küçük harfe dönüştürür.
- Girilen karakterin küçük halini döndürür, bu değişkene atanabilir.

```
char kelime[2]={'A'} //Null ve harf için toplam 2 elemanlık yer açıldı.
char lower[]=tolower(kelime[0]); //lower[0]='a'
```

toupper()

- Karakteri büyük harfe dönüştürür.
- Girilen karakterin büyük halini döndürür, bu değer değişkene atanabilir.

```
char kelime[2]={'a'} //Null ve harf için toplam 2 elemanlık yer açıldı.
char up[]=touppper(kelime[0]); //up[0]='A'
```

Gösterge Dizileri

- Bu diziler içinde pointer tutan dizilerdir.
- Genellikle içinde dizi tutarlar yani dizi tutan dizilerdir. (String vs.)
- Dizi tanımının başına asteriks koyarak oluşturulur.

```
char *names[]={"Recep","Emre","Ali","Hakan"}

//names[0]=Recep
//names[1]=Emre
//names[2]=Ali
//names[3]=Hakan
```

- Yukarıdaki names isimli göstege dizisinin 4 elemanı da pointer'dir ve hepsi gösterdiği string'lerin ilk harfini tutar.

2 Boyutlu Dinamik Dizi

```
int **matris; //2 boyutlu pointer.

matris = (int**)malloc(satir*sizeof(int*));  //Satırlar oluşturuldu.

for(i=0;i<satir;i++){
    matris[i] = (int*)malloc(sutun*sizeof(int));   //Sütunlar oluşturuldu

for(i=0;i<satir;i++){
    free(matris[i]);
}
```

```
free(matris);
```